

2-Sep-2022 Friday

Data Structure Crash Course @ AlgoExpert

Videos 1-6 Introductory, Rest Course

Video 1: Introduction

DS- Tools to problem Solving.

Video 2: What are Data Structures?

DS would be a collection of data values.

Bunch of values that are related to each other and are manipulated by certain instructions, hence they have fancy names.

One should not get overwhelmed with their names have been entitled.

Video 3: Complexity Analysis

A problem might have multiple solutions.

Complexity: Solution not being difficult/lengthy

Complexity $\begin{cases} \nearrow \text{Time} \\ \searrow \text{Space} \end{cases}$] Space-Time Complexity

Solution depends on use-case.
Space-time complexity hence changes.

Video 4: Memory

3rd Sep '22

Saturday

3 Key Terms.

Complexity Analysis:

The process of determining how efficient an algorithm is. It usually involves finding both the time complexity and the space complexity of an algorithm.

Complexity analysis is effectively used to determine how "good" an algorithm is and whether it's better than another one.

Time

Complexity

◦ A measure of how fast an algorithm runs, time complexity is a central concept in the field of algorithms and in coding interviews.

It is expressed using Big O Notation.

Space

Complexity

◦ A measure of how much auxiliary memory an algorithm takes up, space complexity is a central concept in the field of algorithms and in coding interviews.

It's expressed using Big O Notation.

Video 4: Memory

1 memory slot can store = 1 byte or 8 bits.

Imp.

Binary Nos.
(5 mins. wikipedia
article).

8 bits: (1) 0000 0001
(2) 0000 0010
etc.

Imp.

endianness

we store most
significant byte on
the left in a memory
location.

8 bits
or

1 byte

in one memory slot.

7 6 5 4 3 2 1
 $2^8 = 256$ data
value

Int bit is a fixed length
hence no. of bytes
it will require in
memory are determined
then.

can
be
contained

$$32 \text{ bits} = \frac{32 \times 4}{8} =$$

4 memory
slots.

Key Terms

Bit :

Short for binary
digit, a bit is a
fundamental unit of
information in computer
science that represents
a state with one of
two values, typically
0 and 1.

$$64 \text{ bits} = \frac{64 \times 8}{8} =$$

8 memory
slots.

Byte: A group of eight bits.

For eg. 0110 1000 is a byte

A single byte can represent up to 256 data values $2^8 = 256$.

Since, a binary no. is a number expressed with only two symbols, like 0 and 1, a byte can effectively represent all of the nos between 0 and 255, inclusive, in binary format.

The following bytes represent the numbers 1, 2, 3 and 4 in binary format.

1	0000	0001
2	0000	0010
3	0000	0011
4	0000	0100

Fixed Width Integer:

An integer represented by a fixed amount of bits.

For ex. a 32-bit integer is an integer represented by 32 bits (4 bytes) and a 64-bit integer represented by 64 bits (8 bytes).

The following is the 32-bit representation of the no. 1, with clearly separated bytes.

00000000 00000000 00000000 00000001

The following is the 64-bit representation of the no. 10 with clearly separated⁻ⁿ bytes.

00000000 00000000 00000000 00000000

00000000 00000000 00000000 00001010

Regardless of how large an integer is, its fixed-length-integer representation is, by definition, made up of a constant number of bits.

It follows that, regardless of how large an integer is, an operation performed on its fixed-width-integer representation consists of a constant no. of bit manipulations, since the ~~number~~ integer is made up of a fixed number of bits.

Memory: Broadly speaking, memory is the foundational layer of computing, where all data is stored.

In the context of coding interviews, its important to note the following points.

P.T.O.

- Data stored in memory is stored in bytes and, by extension bits.
- Bytes in memory can "point" to other bytes in memory, so as to store references to other data.
- The amount of memory that a machine has is bounded, making it valuable to limit how much memory an algorithm takes up.
- Accessing a byte or a fixed no. of bytes (like 4 bytes or 8 bytes in the case of 32-bit and 64-bit integers) is an elementary operation, which can be loosely treated as a single unit of operational work.

Video 5 : Big O-Notation

Speed of the algorithm was dependent upon the size of the datastructure (eg. Array).

* Time Complexity → How fast an algorithm runs as the size of the input is changed (inc./dec.)

Notation used to denote is:-

$O(n)$, where n is the input size.

Asymptotic Analysis - Behaviour of an algorithm as the Complexity $\uparrow / \downarrow$.

$O(1)$ has not unit and is rather an abstract value. and it constitutes rather as an elementary operation.

In memory, let's say for int type of data, inside every time 8 elementary operations are taking place. $\therefore$ in memory we have 8 bits / 1 byte ~~an~~ per memory slot / address multiplied by 4 bytes for int data.

Increasing order of Complexity:-

$O(1)$ - Constant

$O(\log(N))$ - Logarithmic

$O(N)$ - Linear

$O(N^2)$ - Quadratic

$O(N^3)$ - Cubic

$O(N^4)$ -

$O(2^N)$ - Exponential

$O(N!)$ - Factorial

Come into same class.

$O(n \log n)$ $\times$
 $\downarrow$
Log-Linear

If you have 2 different variables in a Big O Notation, both of them remain.

Video Notes :

Big O Notation

The notation used to describe the time complexity and space complexity of algorithms.

Variables used in Big-O Notation denote the sizes of inputs to algorithm.

For eg. $O(N)$ might be time complexity of an algorithm that traverses through an array of length N .

Similarly, $O(n+m)$ might be the time complexity of an algorithm that traverses through an array of length n and through a string of length m .

The following are examples of common complexities and their Big O notations, ordered from fastest to slowest:

1. Constant :	$O(1)$
2. Logarithmic :	$O(\log(n))$
3. Linear :	$O(n)$
4. Log-Linear :	$O(n \log(n))$
5. Quadratic :	$O(n^2)$
6. Cubic :	$O(n^3)$
7. Exponential :	$O(2^n)$
8. Factorial :	$O(n!)$

Note, that in the context of coding interviews, Big O notation is usually understood to describe the worst case complexity of an algorithm, even though the worst-case complexity might differ from the average-case complexity.

For eg., some sorting algorithms have different time complexities depending on the layout of elements in their input array.

In rare cases, their time complexities will be much worse than in more common cases. Similarly, an algorithm that takes in a string and performs special operations on uppercase letters might

have a different time complexity when run on an input string of only uppercase characters vs. on ~~the~~ an input string with just a few uppercase characters.

Thus, when describing the time complexity of an algorithm, it can sometimes be helpful to specify whether the time complexity refers to the average case or to the worst case (eg. "this algorithm runs in $O(n \log n)$ time on average and in $O(n^2)$ time in the worst case").

Video 6: Logarithm

$$\log_b(x) = y \text{ iff } b^y = x$$

$$\log_2(x) = y \text{ iff } 2^y = x$$

or

$$\log_2(N) = x \text{ iff } 2^x = N$$

$$\text{eg. } \log(1) = 0.$$

$$\therefore 2^0 = 1.$$

$$\log_2(4) = ?$$

$$2^x = 4.$$

$$\Downarrow$$
$$2^2 = 4.$$

$$\text{So, } \log_2(4) = 2.$$

$$\log(16) = x.$$

$$2^x = 16.$$

$$2^4 = 16 \therefore x = 4. \log(16) = 4.$$

$$2^4 = 16$$

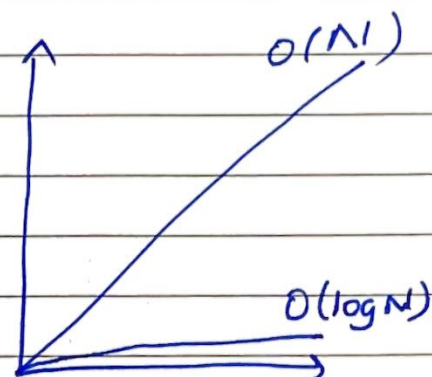


$$2^{(2^2)} = 32 \quad (16 + 16)$$

$$2^{20} = 1M$$

$$2^{30} = 1B.$$

$$O(\log N) < O(N)$$



* If the input doubles in size, the operations performed only increase by 1.

Key Terms

1.) Logarithm:-

A mathematical concept that's widely used in Computer-Science and that's defined by the following equation:-

$$\log_b(x) = y \quad \text{iff} \quad b^y = x.$$

In the context of coding interview the logarithm is used to describe the complexity analysis of algorithms, and its usage always implies a logarithm.

of base-2. In other words, the logarithm used in the context of coding interview is defined by the following equation:

$$\log(n) = y \iff 2^y = n$$

In plain English, if an algorithm has a logarithmic time complexity $O(\log(n))$, where n is the size of the input, then whenever the algorithm's input doubles in size (ie, whenever n doubles) the number of operations needed to complete the algorithm only increases by one unit. Conversely, an algorithm with a linear time complexity would see its number of operations double if its input size doubled.

As an example, a linear-time-complexity algorithm with an input of size 1,000 might take roughly 1,000 operations to complete, whereas a logarithmic-time-complexity algorithm with the same i/p would take roughly 10 operations to complete, since $2^{10} = 1,000$.

Video 7: Arrays

[1, 2, 3]



1 byte = 8 bits.

$$8 \cdot 3 = 24 \text{ bits.}$$

* Static & Dynamic Arrays.

	3	4	5
6	7	8	9
10	11	12	13
14	15	16	17
18	19	20	21
22	23	24	25
		24	25
26			

Static Arrays.

get = $O(1)$ S&T

set = $O(1)$ S&T

init = $O(N)$ S&T

traverse = $O(N)$ T

~~$O(1)$ S~~ ~~traverse~~

copy = $O(N)$ S&T

insert = $O(N)$ T

$O(1)$ S

C++ Java
↓

Dynamic Array (Vectors Array List)

Python, JavaScript ([] ← List)

Dynamic Array

of memory.

$O(1)$

Allocation in the beginning is initially double the size of insertion.

$O(2)$

After exhaustion, ~~after~~ of all the extra space, new memory is allocated at a new slot in the memory, ~~after~~ again double the size of the array then.

$O(4)$

Copy operation happens everytime the space exhausted.

✓ $O(N)$ double / 2-times → amortized Analysis / case
Append operations / Insert ⇒

$$N + \frac{N}{2} + \frac{N}{4} + \frac{N}{8} \dots 1$$

Key Terms

Array - A linear collection of data values that are accessible at numbered indices, starting at index 0.

A static array is an implementation of an array that allocates a fixed amount of memory to be used for storing the array's values. Appending values to the array therefore involves copying the entire array and allocating new memory of it, accounting for the extra space needed for the newly appended value. This is a linear time operation $O(n)$.

A dynamic array is an implementation of an array that preemptively allocates double the amount of memory needed to store the array's value. Appending values to the array is a constant-time operation until the allocated memory is filled up, at which point the array is copied and double the memory is once again allocated for it. This

implementation leads to an amortized constant-time insertion-at-end operation.

A lot of popular programming languages like JavaScript and Python implement arrays as dynamic arrays.

The following are an array's standard operations and their corresponding time complexities:

operation	Complexity
1. Accessing a value at a given index	$O(1)$ Constant
2. Updating a value at a given index	$O(1)$ Constant
3. Inserting a value at the a) beginning	$O(N)$ Linear
b) middle	$O(N)$ Linear
4. Inserting c) end	
i) dynamic	amortized constant($O(1)$)
ii) static	$O(N)$ Linear

operation	Complexity
4. <u>Removing a value at the</u>	
i) beginning	$O(N)$ Linear
ii) middle	$O(N)$ "
iii) end	$O(1)$ Constant
5. <u>Copying the array</u>	$O(N)$ Linear

Video 8: Linked Lists.

Singly
LL

get : $O(i)$ T & $O(1)$ S

set : $O(i)$ T & $O(1)$ S

init : $O(N)$ S & T

copy : $O(N)$ S & T

traverse : $O(N)$ T & $O(1)$ S.

insertion :

begin		-
middle		-
and		-

deletion :

begin		-
middle		-
and		-

Video 8: Linked Lists

Key Terms

1) Singly Linked List

A data structure that consists of nodes, each with some value and a pointer to the next node in the linked list. A linked list node's value and next node are typically stored in value and next properties respectively,

The first node in a linked list is referred to as the head of the linked list, while the last node in a linked list, whose next property points to the null value is known as the tail of the linked list.

The visual representation of a singly linked list whose nodes hold integer values :-

0 → 1 → 2 → 3 → 4 → 5 → null

A singly linked list typically exposes its head to its user for easy access. While finding a node in a singly linked list

Involves traversing through all of the nodes leading up to the node in question (as opposed to instant access with an array), adding or removing nodes simply involves overwriting next pointers (assuming that you have access to the node right before the node that you're adding or removing).

2) Doubly Linked List

Similar to a Singly Linked List, except that each node in a doubly linked list also has a pointer to the previous node in the linked list. The previous node is typically stored in a prev property.

Just as the next property of a doubly linked list's tail points to the null value, so too does the prev property of a doubly linked list's head.

A visual rep. of a doubly linked list whose nodes hold integer values:

null $\longleftrightarrow$ 0 $\longleftrightarrow$ 1 $\longleftrightarrow$ 2 $\longleftrightarrow$ 3 $\longleftrightarrow$ 4 $\longleftrightarrow$ 5 $\longleftrightarrow$ null

while a doubly linked list typically exposes both its head and tail to its user, as opposed to just its head in the case of a singly linked list. It otherwise behaves very similar to a singly linked list.

3) Circular linked list

A linked list has no clear head or tail, because its "tail" points to its "head", effectively forming a closed circle.

A circular linked list can be either a singly linked circular linked list or a doubly circular linked list.

SLL

DLL

Accessing		
head	$O(1)$	$O(1)$
middle node	$O(n)$	$O(n)$
tail	$O(n)$	$O(1)$
Inserting / Removing		
head	$O(1)$	$O(1)$
middle node	$O(n)$ to access + $O(1)$	$O(n)$ to access + $O(1)$
tail	$O(n)$ to access + $O(1)$	$O(1)$
Searching for a value.	$O(n)$	$O(n)$

Video 9: Hash Table

A data structure that provides fast insertion, deletion and lookup of key/value pairs.

Under the hood, a hash table uses a dynamic array of linked lists to efficiently store key/value pairs. When inserting a key/value pair, a hash table function maps the key, which is typically a string (or any data that can be hashed, depending on the implementation of the hash table), to an integer value and, by extension, to an index in the underlying dynamic array. Then, the value associated with the key is added to the linked list stored at the index in the dynamic array, and a reference to the key is also stored with the value.

Hash tables rely on highly optimized hash functions to minimize the number of collisions that occur when storing values: cases where two keys map to the same index.

Below is an example of what a hash table might look like under the hood:

[
0 : (value1, key1) $\rightarrow$ null
1 : (value2, key2) $\rightarrow$ (value3, key3) $\rightarrow$ (value4, key4)
2 : (value5, key5) $\rightarrow$ null
3 : (value6, key6) $\rightarrow$ null
4 : null
5 : (value7, key7) $\rightarrow$ (value8, key8)
6 : (value9, key9) $\rightarrow$ null
]

In the hash table above, the keys key2, key3 & key4 collided by all being hashed to 1, and the keys key7 and key8 collided by both being hashed to 5.

The following are ~~the~~ a hash table standard operations and their corresponding time complexities:

- Inserting a key/value pair :
 - $O(1)$ on average
 - $O(n)$ in the worst case.
- Removing a key/value pair :
 - $O(1)$ on average
 - $O(n)$ in the worst case

- Looking up a key:
 $O(1)$ on average
 $O(n)$ in the worst case

The worst-case linear-time operations occur when a hash table experiences a lot of collisions leading to long linked ~~for~~ lists internally, which take $O(n)$ time to traverse.

However, in practice and especially in coding interviews, we typically assume that the hash functions employed by hash tables are so optimized that collisions are extremely rare and constant-time operations are all but guaranteed.

Video 10: Stacks and Queues

Stack: An array-like data structure whose elements follow the LIFO rule: Last In, First Out.

A stack is often compared to a stack of books on a table: the last book that's placed on the stack of books is the first one that's taken off the stack.

The following are a stack's standard operations and their corresponding time complexities.

- Pushing an element onto the stack = $O(1)$ constant S&T
- Popping an element off the stack = $O(1)$ constant S&T
- Peeking the element on top of the stack = $O(1)$ constant
- Searching for an element in the stack = $O(n)$ linear time

A stack is typically implemented with a dynamic array or with a singly linked list.

Queue - An array-like data structure whose elements follow the FIFO rule = First In First Out

A queue is often compared to a group of people standing in line to purchase items at a store. The first person to get in line is the first one to purchase items and get out of the queue.

The following are a queue's standard operations and their corresponding time complexities:

- Enqueuing an element into the queue = $O(1)$ Constant
- Dequeuing an element out of the queue = $O(1)$ Constant
- Peeking at the element at the front of the queue = $O(1)$ Constant
- Searching for an element in the queue = $O(n)$ Linear time

A queue is typically implemented with a doubly linked list.

- * peek doesn't remove the current element, it just skips it.

Video 11: String

One of the fundamental data types in Computer Science, strings are stored in memory as arrays of integers, where each character in a given string is mapped to an integer via some character encoding standard like ASCII.

Strings behave much like normal arrays, with the main distinction being that, in more programming

languages (C++ is a notable exception). Strings are immutable, meaning that they can't be edited after creation. This also means that simple operations like appending a character to a string are more expensive than they might appear.

The canonical example of an operation that's deceptively expensive due to string immutability is the following:

```
string = "this is a string"  
newString = string + "!"
```

```
for character in string:  
    newString += character
```

The operation above has a time complexity of $O(n^2)$ where n is the length of string, because each addition of a character to newString creates an entirely new string and is itself an $O(n)$ operation.

Therefore, n $O(n)$ operations are performed, leading to an $O(n^2)$ time-complexity operation overall.

	space	Time
Traverse	$O(N)$	$O(1)$
Copy	$O(N)$	$O(N)$
get	$O(1)$	$O(1)$

Video 12: Graph

Graph: A collection of nodes or values called vertices that might be related; relations b/w vertices are called edges.

Many things in life can be represented by graphs; for example, a social network can be represented by a graph whose vertices are users and whose edges are friendships b/w the users. Similarly, a city map can be represented by a graph whose vertices are locations in the city and whose edges are roads b/w the locations.

Graph Cycle: Simply put, a cycle occurs in a graph when three or more vertices in the graph are connected so as to form a closed group loop.

NOTE: The definition of a graph cycle is sometimes broadened to include cycles of length two or one; in the context of coding interviews, when dealing with questions that involve graph cycles, it's important to clarify what exactly constitutes a cycle.

Acyclic Graph: - A graph that has no cycles.

Cyclic Graph: - A graph that has at least one cycle.

Directed Graph: A graph whose edges are directed, meaning that they can only be traversed in one direction, which is specified.

For example, a graph of airports and flights would likely be directed, since a flight specifically goes from one airport to another (i.e., it has a direction without necessarily implying the presence of a flight in the opposite direction).

Undirected Graph: A graph whose edges are undirected, meaning that they can be traversed in both directions.

For eg., a graph of friends would likely be undirected, since a friendship is, by nature, bidirectional.

Connected Graph: A graph is connected if for every pair of vertices in the graph, there's a path of one or more edges connecting the given vertices.

In the case of a directed graph, the graph is:

Strongly connected: if there are bidirectional connections b/w the vertices of every pair of vertices (ie, for every vertex-pair (u, v) you can reach v from u and u from v).

Weakly connected: if there are connecting (but not necessarily bidirectional ones) b/w the vertices of every pair of vertices.

A graph that isn't connected is said to be disconnected.

Storing a graph = $O(V+E)$

Traversing = DFS, BFS

Video 13: Trees

- 1) Tree: A data structure that consists of nodes, each with some value and pointers to child-nodes, which recursively form subtrees in the trees.

The first node in a tree is referred to as the root of the tree, while the nodes at the bottom of a tree (the nodes with no child-nodes) are referred to as leaf nodes or simply leaves. The paths between the root of a tree and its leaves are called branches, and the height of a tree is the length of its longest branch. The depth of a tree node is its distance from its tree's root; this is also known as the node's level in the tree.

A tree is effectively a graph that's connected, directed, and acyclic, that has an explicit root node, and whose nodes all have a single parent (except for the root node, which effectively

has no parent). Note that in most implementations of trees, tree nodes don't have a pointer to their parent, but they can if desired.

There are many types of trees and tree-like structures, including ~~trees~~, binary trees, heaps, and piles.

Binary Tree:

A tree whose nodes have up to two child-nodes.

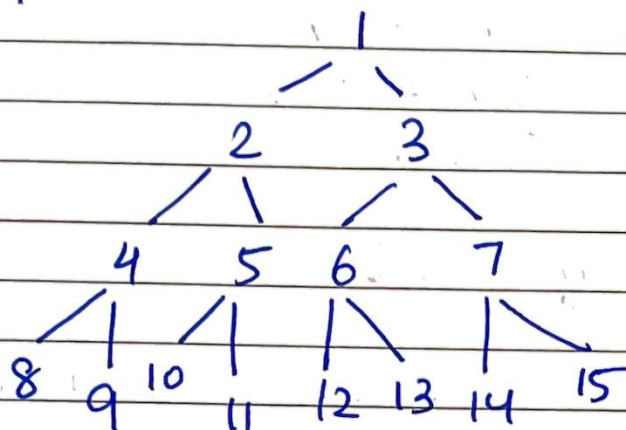
The structure of a binary tree is such that many of its operations have a logarithmic time complexity, making the binary tree an incredible attractive and commonly used data structure.

K-ary Tree

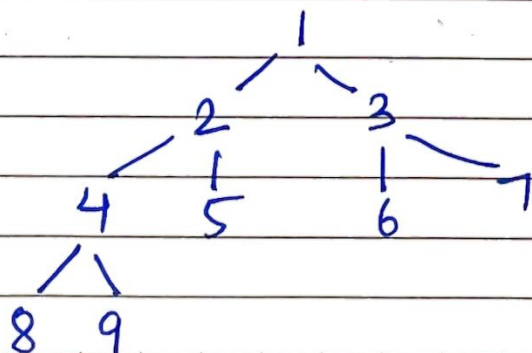
A tree whose nodes have up to K -child-nodes. A binary tree is a K -ary tree where $K=2$.

Perfect Binary Tree: A binary tree whose interior nodes all have two child-nodes and whose leaf nodes all have the same depth.

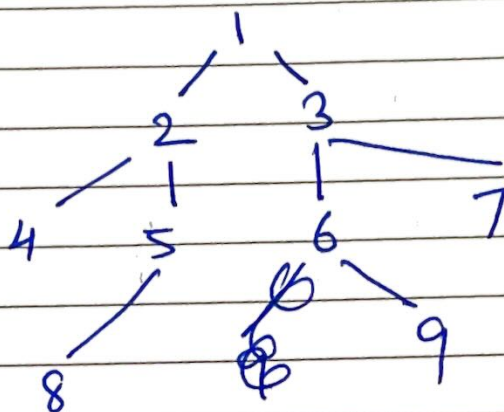
Example:



Complete Binary Tree: A binary tree that's almost perfect; its interior nodes all have two child-nodes, but its leaf nodes don't necessarily all have the same depth. Furthermore, the nodes in the last level of a complete binary tree are as far left as possible. Example



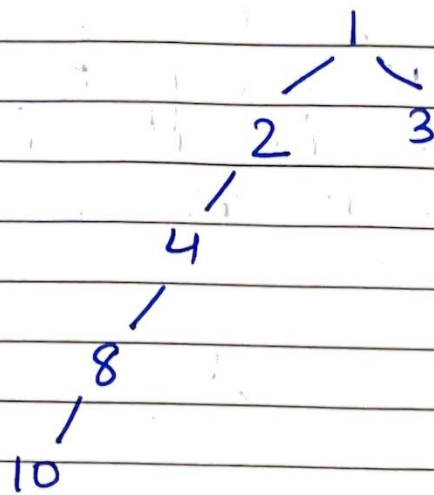
Conversely, the following binary tree isn't complete, because the nodes in its last level aren't as far left as possible:



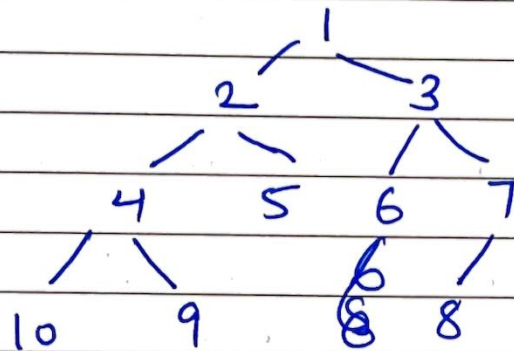
Balanced Binary Tree ? A binary tree whose nodes all have left and right subtrees whose heights differ by no more than 1.

A balanced binary tree is such that the logarithmic time complexity of its operations is maintained.

For example: inserting a node at the bottom of the following imbalanced binary tree's left subtree would clearly not be a logarithmic-time operation, since it would involve traversing through most of the tree's nodes.



The following is an example of a balanced binary tree:



Full Binary Tree: A binary tree whose nodes all have either two child-nodes or zero child-nodes.

Example:

